

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: MLA03

COURSE NAME: Reinforcement learning for Environment and Agent

CO5: Implement policy-based reinforcement learning methods to develop efficient solutions for complex environments. (BL2, BL3, BL6)

Assessment Tool Weightage: 35%

Dr G A. SENTHIL

QUESTIONS: 10

Total Marks: 50

Code of Conduct:

I __T Bhanu Teja(192472259)__certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Bhanu Teja

Annexure A

Mock Interview

1. Hierarchical Reinforcement Learning (HRL) – Autonomous Warehouse Navigation

- **AIM**

To simulate a Hierarchical Reinforcement Learning agent for autonomous warehouse navigation by dividing the navigation problem into smaller sub-tasks.

- **ABSTRACT**

Hierarchical Reinforcement Learning divides a complex task into high-level and low-level tasks. In a warehouse, the high-level agent decides the destination, such as selecting a storage location, while the low-level agent decides individual movements such as moving up, down, left, or right. This reduces the complexity of learning and helps the robot reach its destination efficiently.

- **ALGORITHM**

1. Create a warehouse grid environment.
2. Define the robot's initial position and target position.
3. Create a high-level controller to select the sub-goal.
4. Create a low-level controller to select movement actions.
5. Move the robot toward the selected sub-goal.
6. Provide a positive reward when the sub-goal is reached.
7. Provide a negative reward for obstacles or invalid movements.
8. Continue until the final destination is reached.
9. Display the learned navigation path.

□ OUTPUT:-

```
1 import numpy as np
2
3 warehouse = np.zeros((5, 5))
4
5 robot = [0, 0]
6 goal = [4, 4]
7
8 # High-level controller
9 def select_subgoal():
10     return [2, 2]
11
12 # Low-level controller
13 def move_robot(robot, subgoal):
14
15     if robot[0] < subgoal[0]:
16         robot[0] += 1
17     elif robot[1] < subgoal[1]:
18         robot[1] += 1
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
```

Robot Position: [2, 2]
Sub-goal Reached!
Robot Position: [3, 2]
Robot Position: [4, 2]
Robot Position: [4, 3]
Robot Position: [4, 4]
Final Destination Reached!

CONCLUSION:-

The HRL agent successfully divides warehouse navigation into high-level and low-level tasks. This hierarchical approach simplifies complex navigation and helps the robot reach its destination efficiently.

2. Hierarchical Abstract Machine (HAM) – Robot Sequential TasksAIM

To implement a Hierarchical Abstract Machine for controlling a robot that performs sequential tasks such as moving, picking an object, and delivering it.

ABSTRACT

A Hierarchical Abstract Machine represents robot behavior using a hierarchy of tasks and actions. A high-level task is divided into smaller sequential operations. In this project, the robot first moves to an object, picks it up, moves to the delivery location, and finally drops the object.

ALGORITHM

1. Define the main robot task.
2. Divide the main task into smaller subtasks.
3. Move the robot to the pickup location.
4. Perform the pickup action.
5. Move to the delivery location.
6. Perform the drop action.
7. Check whether all tasks are completed.
8. Display the execution sequence.

OUTPUT:-

```
1 class RobotHAM:
2
3     def move(self, location):
4         print("Robot moving to:", location)
5
6     def pick(self):
7         print("Robot picked the object")
8
9     def drop(self):
10         print("Robot dropped the object")
11
12     def execute(self):
13
14         print("Main Task Started")
15
16         # Sub-task 1
17         self.move("Pickup Location")
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
```

Problems Output Debug Console Terminal Ports ... Python + - ...

```
anchigunashekar/OneDrive/Desktop/python-RL/C05-2.PY
Main Task Started
Robot moving to: Pickup Location
Robot picked the object
Robot moving to: Delivery Location
Robot dropped the object
All Tasks Completed
```

CONCLUSION:-

The HAM successfully controls the robot using hierarchical and sequential tasks. Breaking the main task into smaller actions makes robot control easier to design, understand, and execute.

3. MAXQ Value Decomposition – Taxi Navigation

□ AIM

To develop a simplified MAXQ Value Decomposition algorithm for a taxi navigation environment.

□ ABSTRACT

MAXQ is a hierarchical reinforcement learning algorithm that decomposes a complex task into smaller subtasks. In a taxi environment, the main task of transporting a passenger can be divided into subtasks such as navigating to the passenger, picking up the passenger, navigating to the destination, and dropping off the passenger.

ALGORITHM

1. Define the taxi environment.
2. Define the main taxi task.
3. Divide the task into subtasks.
4. Calculate the value of each subtask.
5. Select the subtask with the highest value.
6. Execute the selected subtask.
7. Update the value of the subtask.
8. Repeat until the passenger reaches the destination.
9. Display the learned values.

OUTPUT:-

```
1 import numpy as np
2
3 # MAXQ value table
4 Q = {
5     "Navigate_Passenger": 0,
6     "Pickup": 0,
7     "Navigate_Destination": 0,
8     "Dropoff": 0
9 }
10
11 rewards = {
12     "Navigate_Passenger": 5,
13     "Pickup": 10,
14     "Navigate_Destination": 8,
15     "Dropoff": 15
16 }
17
18 alpha = 0.1
```

Problems Output Debug Console **Terminal** Ports ... Python + v ... |

```
MAXQ Subtask Values:
Navigate_Passenger : 4.97
Pickup : 9.95
Navigate_Destination : 7.96
Dropoff : 14.92

Highest Value Subtask: Dropoff
```

CONCLUSION:-

The MAXQ approach decomposes taxi navigation into smaller subtasks and learns the value of each task independently. This hierarchical decomposition makes complex navigation problems easier to solve.

4. Meta-Reinforcement Learning – Multiple GridWorld Environments

□ AIM

To simulate a Meta-Reinforcement Learning agent that rapidly adapts to different GridWorld environments.

ABSTRACT

Meta-Reinforcement Learning enables an agent to learn how to learn across multiple tasks. In this project, the agent is trained on different GridWorld environments with different goal locations. Instead of learning each environment completely from the beginning, the agent uses previous knowledge to adapt quickly to a new environment.

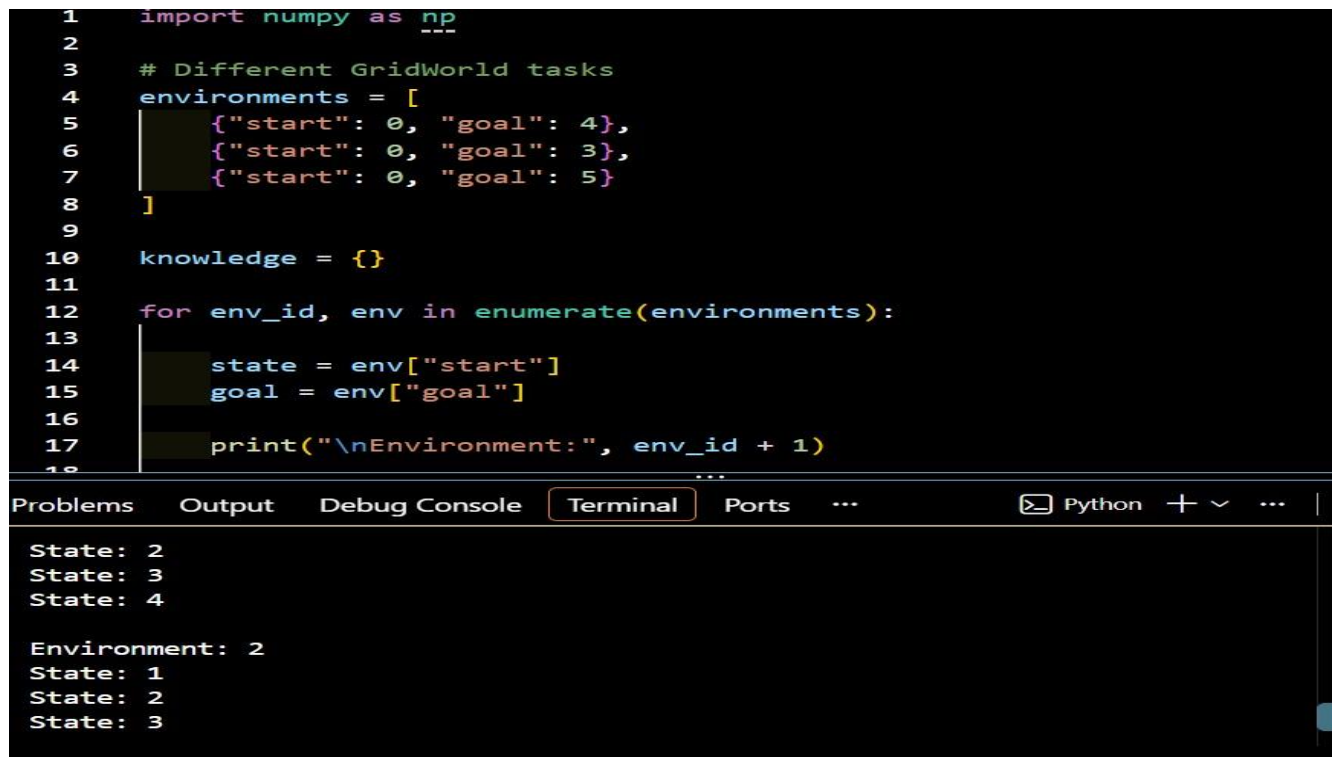
ALGORITHM

1. Create multiple GridWorld environments.
2. Assign different goal positions to each environment.
3. Initialize shared knowledge for the agent.
4. Train the agent on multiple environments.

5. Store the learned information.
6. Introduce a new GridWorld environment.
7. Use previous knowledge to select actions.
8. Adapt the policy using a small number of steps.
9. Measure adaptation performance.
10. Compare learning speed across environments.

OUTPUT:-

```
1 import numpy as np
2
3 # Different GridWorld tasks
4 environments = [
5     {"start": 0, "goal": 4},
6     {"start": 0, "goal": 3},
7     {"start": 0, "goal": 5}
8 ]
9
10 knowledge = {}
11
12 for env_id, env in enumerate(environments):
13
14     state = env["start"]
15     goal = env["goal"]
16
17     print("\nEnvironment:", env_id + 1)
18
```



The image shows a code editor with a dark theme. The code defines a list of GridWorld environments with different start and goal states. It then iterates over these environments, printing the state and goal for each. The output in the terminal window shows the state and goal for each environment, with the state being 0 and the goal being 4, 3, and 5 respectively.

CONCLUSION:-

The Meta-RL agent uses knowledge obtained from multiple GridWorld tasks to improve adaptation to new environments. This allows the agent to learn new tasks faster than learning each task independently.

5. Multi-Agent Reinforcement Learning (MARL) – Cooperative Robot Path Planning

AIM

To design a Multi-Agent Reinforcement Learning system in which multiple robots cooperate to find efficient paths while avoiding collisions.

ABSTRACT

Multi-Agent Reinforcement Learning involves multiple agents learning and interacting within the same environment. In cooperative robot path planning, each robot has an individual starting position and destination. The robots share information and rewards to reach their destinations while avoiding collisions and unnecessary movements.

ALGORITHM

1. Create a grid environment.
2. Initialize multiple robot agents.
3. Define the starting and goal positions.
4. Allow each robot to select an action.
5. Move the robots simultaneously.
6. Give positive rewards for reaching goals.
7. Give negative rewards for collisions.
8. Update each robot's policy.
9. Repeat the process for multiple episodes.
10. Evaluate the final paths of all robots.

OUTPUT:-

```
20         position[1] += 1
21     elif position[1] > goal[1]:
22         position[1] -= 1
23     return position
24
25 for step in range(10):
26     positions = []
27     for name, robot in robots.items():
28         robot["position"] = move(robot)
29     positions.append(tuple(robot["position"]))
30
31 Robot_2 Position: [0, 4]
32 ---
33 Robot_1 Position: [4, 4]
34 Robot_2 Position: [0, 4]
35 ---
36 Final Robot Positions:
37 Robot_1 : [4, 4]
38 Robot_2 : [0, 4]
```

CONCLUSION

The MARL system allows multiple robots to learn and perform path planning cooperatively. By considering other robots' positions and using rewards and penalties, the system can reduce collisions and improve overall navigation efficiency.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Technical Knowledge	25	Demonstrates strong and accurate subject knowledge with in-depth understanding	Shows good knowledge with minor gaps	Basic knowledge with noticeable errors	Lacks fundamental technical knowledge
Problem Solving	25	Solves problems logically and applies concepts effectively in real scenarios	Applies concepts with moderate accuracy	Limited problemsolving ability; requires guidance	Unable to solve or apply concepts
Analytical Thinking	15	Analyzes questions critically and provides wellstructured, logical answers	Provides reasonable analysis with some structure	Superficial or partially structured responses	No analytical thinking demonstrated
Communication	15	Communicates clearly, confidently, and professionally with proper terminology	Communicates well with minor hesitation	Communication lacks clarity or confidence	Unable to communicate effectively
Confidence	15	Displays high confidence, positive attitude, and professional behavior throughout	Shows good confidence with minor issues	Low confidence; inconsistent behavior	Lacks confidence and professionalism
Response to Questions	10	Responds accurately, promptly, and elaborates with relevant examples	Responds correctly but with limited depth	Partial responses; needs prompting	Unable to respond appropriately